Desarrollo Web en Entorno Cliente

Inyección de dependencias, formularios y validaciones

Índice

Esquema 3

Material de estudio 4

7.1. Introducción y objetivos 4

7.2. ¿Qué es y para qué se usa la inyección de dependencias? 6

7.3. Ejemplo de inyección de dependencia 8

7.4. Formulario de tipo Template. FormModule 11

7.5. Formulario de tipo Model. Reactive Forms 14

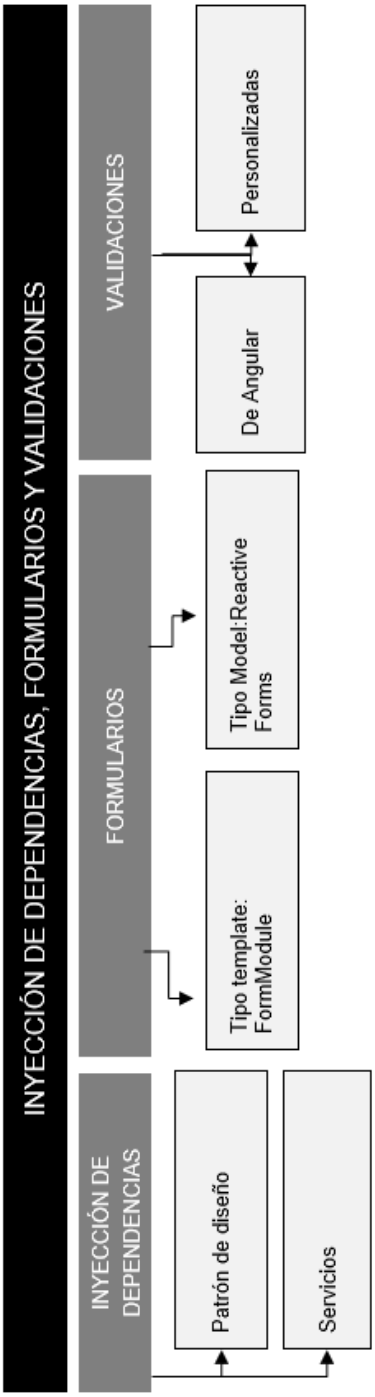
7.6. Validaciones en Angular y validaciones personalizadas 17

A fondo 23

Entrenamientos 24

Test 30

Esquema


INYECCIÓN DE DEPENDENCIAS, FORMULARIOS Y VALIDACIONES
INYECCIÓN DE DEPENDENCIAS
Patrón de diseño
Servicios
FORMULARIOS
Tipo template: FormModule
Tipo Model:Reactive Forms
VALIDACIONES
De Angular
Personalizadas

Material de estudio

7.1. Introducción y objetivos

Este es un tema que se divide en dos partes muy diferenciadas. Inyección de dependencias y formularios.

El objetivo que intentamos cumplir dentro al ver la inyección de dependencias no es otro que entender como es este patrón de diseño para que no nos resulte extraño cuando nos lo encontremos en temas posteriores. Explicaremos, como se usa una dependencia y cómo podemos aislarlas de nuestras clases para que estas sean más fuertes, flexibles y escalables.

Por otro lado, las dos acciones fundamentales que cumple cualquier aplicación web son:

- Mostrar datos al cliente.
- Recoger datos por parte del cliente.

El objetivo de la segunda parte de este tema es entender cómo funcionan los formularios en Angular como herramienta para recoger información que el usuario puede introducir en nuestras aplicaciones, ya sea para registrar datos o hacer un proceso de login, etc.

El elemento básico para la recuperación de estos datos dentro de una webapp son los formularios y conocerlos bien y ver qué posibilidades tienen es fundamental para realizar buenas aplicaciones que permitan al usuario interactuar con la propia aplicación.

Hasta ahora solo conocíamos una forma de recoger datos de un usuario que era con el Two Way Data Biding, a la que a cualquier input le asociábamos una propiedad de

nuestra clase del componente y automáticamente se llenaba con los datos que el usuario introducía.

Esto está muy bien, pero es limitado en cuando a funcionalidad se refiere. Por esta razón los formularios son parte fundamental del desarrollo de una aplicación ya que cumplen ciertas funcionalidades.

- Recogen los datos por parte del usuario
- Controlan los cambios que se produzcan en los propios campos del formulario
- Deben validar dichos campos.
- Es importante que muestren los posibles errores derivados de los posibles datos introducidos

Angular posee un módulo especial para trabajar con todo lo relacionado a los formularios. Todas las funcionalidades vistas anteriormente ya se encuentran implementadas dentro de este módulo. Posee una gran variedad de validadores, aparte de la posibilidad de crear los nuestros propios.

Existe la posibilidad también de trabajar con validadores asíncronos que revisen los datos del usuario mientras los está introduciendo. Dicho módulo nos va a permitir trabajar con los campos de nuestros formularios como si fuesen propiedades dentro de un objeto

Existen dos maneras para definir formularios dentro de Angular:

- **Template driven**: son todos aquellos formularios que solo dependen de la plantilla del componente donde lo estemos definiendo.
- **Model driven**: en este caso, la configuración de los diferentes campos del formulario se realiza dentro de la clase del componente asociado. Son la opción más recomendable.

7.2. ¿Qué es y para qué se usa la inyección de dependencias?

La inyección de dependencias es un patrón de diseño muy usado en Angular. Se basa en que las clases no crean sus dependencias por sí mismas, sino que las reciben de fuentes externas.

Por ejemplo, si tienes un componente que depende de un servicio, con el que se comunica con la BBDD o con otros componentes, tal y como explicamos en el tema de inputs y outputs, no creas ese servicio en cada componente donde lo vas a usar, si no que en el constructor del componente solicitas e inyectas el servicio que vas a usar y Angular te lo trae.

Este concepto no solo lo vamos a usar en los servicios si no en otros aspectos de angular como las rutas, por ejemplo.

De esta forma **nuestro código esta desacoplado**, ya que **cada cosa se encarga de lo que debe**, y el componente solo lo consume.

El ejemplo más común de una inyección de dependencias es el servicio que analizaremos más adelante junto con las peticiones externa por HTTP, pero del que ahora haremos una pequeña introducción.

El ejemplo que vamos a ver a continuación con la creación de un servicio viene a explicar lo que es una inyección de dependencia de forma clara. Un componente necesita de un elemento, en este caso un servicio, para funcionar correctamente, así que en lugar de realizar esa funcionalidad dentro del componente la sacamos fuera y así otros componentes la pueden usar a futuro.

Para usar esa dependencia tenemos por supuesto que importarla, pero para poder trabajar con ella debemos pasarla o inyectarla como parámetro dentro de la función constructor de nuestra clase.

Veamos un ejemplo práctico de este interesante concepto teórico para que quede más claro su uso.

Antes de nada, también puntualizar una cosa, una dependencia no tiene que ser solamente un servicio podría ser una función, por ejemplo, o un valor.

Cuando Angular crea una nueva instancia de una clase de componente, determina qué servicios u otras dependencias necesita ese componente al observar los tipos de parámetros del constructor.

Veamos un ejemplo visual:

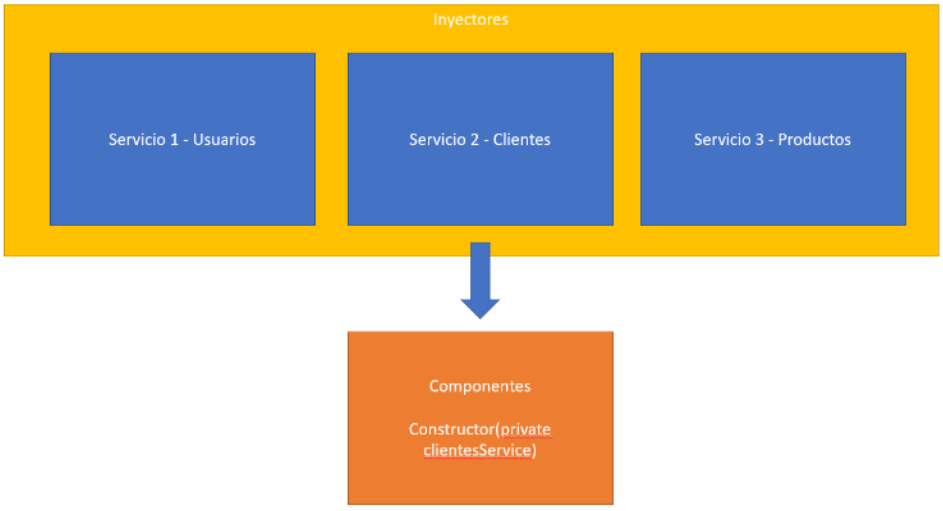


Figura 1. Explicación gráfica de inyección de dependencias. Fuente: contenidos MFPFSTDEV.

7.3. Ejemplo de inyección de dependencia

Lo primero que necesitamos es crear una dependencia que podamos inyectar dentro de un componente.

En este caso y aunque tenemos un tema pensado exclusivamente para la generación de servicios, vamos a usar estos como ejemplo de inyección de dependencias puesto que es el ejemplo más claro dentro de los desarrollos propios que vamos a poder realizar dentro de un proyecto en Angular.

Para crear un servicio al igual que generamos una directiva y un componente tendremos que abrir nuestra terminal y situarnos dentro de la carpeta de proyecto. Una vez allí pondremos el siguiente código dentro en nuestro terminal.

```
ng generate service services/clientes --skip-tests
```

Esto generará dentro de la carpeta de trabajo src/app una carpeta services donde se alojarán los diferentes servicios de nuestra aplicación.

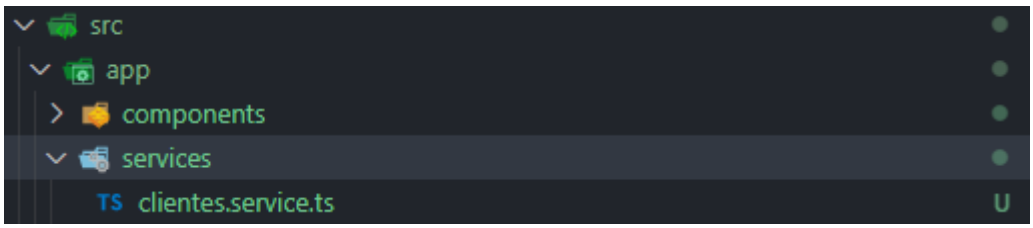


Figura 2. Árbol de carpetas tras la ejecución de la creación del servicio. Fuente: elaboración propia.

El servicio generado, para el ejemplo, va a contener un array de colores que le tenemos que hacer llegar a los diferentes componentes que consulten al servicio, para ello vamos a generar una propiedad privada dentro de la clase, llamada colores de la siguiente forma.

```
@Injectable({
  providedIn: 'root'
})
```

```
export class ClientesService {

  private colores: any[] = ['blue', 'red', 'violet', 'yellow',
'green']
  constructor() { }
```

Hay dos puntos importantes en este código que se ha generado.

- El primer dato importante es que la clase está decorada por la función `@Injectable()`, un decorador que está especificando que este servicio está disponible para el root de la aplicación, es decir para todos los componente que dependan del módulo principal.

- El otro punto importante es que hemos creado un array de colores como **propiedad privada** dentro de la clase con lo que solo es accesible dentro de las funciones de la propia clase.

La única forma que tenemos de dar acceso a esa propiedad a otros componentes es creando un método dentro de esta clase que me retorne el valor de dicha propiedad, lo que **POO** se llama **GETTER**. De la siguiente forma.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ClientesService {

   private colores: any[] = ['blue', 'red', 'violet', 'yellow',
'green']
  constructor() { }

  getColores(): any[]{
    return this.colores;
  }
}
```

Esto que acabamos de crear es un inyectable de clientes que me devuelve un array de colores, que bien podría significar un array de categorías de clientes o una lista de clientes.

Ahora bien **¿cómo podemos usar esto dentro de nuestro componente?** Pues usando la DI (Dependency Ijectable) o inyección de dependencias.

Todo componente que recoja como parámetro de su constructor este servicio podrá usar sus método y propiedades como si estuviesen creadas dentro del propio componente.

Veamos cómo se lleva a cabo esta acción.

```
import { Component, OnInit } from '@angular/core';
import { ClientesService } from 'src/app/services/clientes.service';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})

export class PrimerComponentesComponent implements OnInit {

  arrayColores: any[] = []

  constructor(private clientesServices: ClientesService) {
  }

  ngOnInit(): void {
    this.arrayColores = this.clientesServices.getColores()
  }

}
```

Dentro del componente hemos creado una propiedad para almacenar el array de colores y por ejemplo poderlo pintar dentro del HTML con un `@for`.

Dentro del `constructor()` como argumento privado hemos inyectado el servicio creado previamente y lo hemos importado en la zona de importación arriba de nuestro fichero.

Ahora podemos, por ejemplo llenar el `arrayColores` cuando el componente está cargado, llamando al servicio inyecto previamente y ejecutando el método `getColores()` que habíamos creado en el servicio.

Con esta técnica el servicio puede suministrar esta información a cualquier componente que se la solicite de forma fácil**, simplemente aplicando una inyección de dependencias.**

Los servicios, ya lo veremos más adelante, puede contener multitud de funciones que sirvan datos a diferentes componentes, de esta forma podemos encapsular código reutilizable en cualquier situación y en cualquiera de nuestros componentes.

7.4. Formulario de tipo Template. FormModule

Como ya hemos dicho en la introducción se tratan de módulos que se importan dentro de nuestro modulo principal para que todos nuestros componentes puedan usarlo.

Con lo que el primer paso en la creación de formularios, y poder usar esta potente característica dentro de nuestros componentes es importar el módulo `FormsModule` dentro del `app.module.ts`.

Este módulo es el que me permitía también trabajar con el Two Data Binding y es necesario importarlo para poder usar los formularios de tipo template.

Veamos cómo queda nuestro app-module.ts y como se importa dicho módulo.

```
import { NgModule } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { PrimerComponentesComponent } from './components/primer-componentes/primer-componentes.component';

@NgModule({
  declarations: [
    AppComponent,
    PrimerComponentesComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

En las plantillas donde estemos trabajando, debemos indicar qué formulario vamos a tratar a través de la directiva `ngForm`.

Debemos concretar qué campos son los que vamos a tratar a través de Angular.

En cada campo que vamos a tratar debemos incluir la directiva `ngModel`. Aparte, mediante el evento `ngSubmit` podemos especificar qué método se va a encargar de recibir los datos del formulario cuando el usuario lo envíe.

Veamos un ejemplo de formulario en la parte del HTML del componente:

```
<h1>Formulario</h1>
<form                 novalidate                 #nuevaPersonaForm="ngForm"
(ngSubmit)="onSubmit(nuevaPersonaForm.value)">
  <input type="text" name="nombre" ngModel />
  <button type="submit">Enviar</button>
</form>
```

Cada campo que necesitemos tratar va identificado con la directiva `ngModel` y un atributo `name` con el nombre del campo.

Generamos una nueva variable de plantilla (`#nuevaPersonaForm`) a la cual le asignamos el valor del formulario.

Esta variable nos permite recuperar los datos del formulario dentro del método que maneja dicho formulario.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

  onSubmit(values: any) {
    console.log(values)
  }
}
```

`values` representa un objeto JSON que me devuelve un objeto cuyas claves son los `name` que le hemos puesto a cada uno de los campos de nuestro formulario.

Dentro del método `onSubmit()` puede comprobar que te llegan todos los datos del formulario que acabas de crear como un objeto que posterior mente puede usar para mandar a un servicio, a la base de datos a un array para almacenarlo.

Este tipo de formulario son muy sencillos y me ofrecen de forma fácil y rápida una forma de recoger valores del usuario de la página web.

En este vídeo, *Formulario tipo template*, se trata la creación de un formulario de tipo template. Uso y recogida de datos.

7.5. Formulario de tipo Model. Reactive Forms

En este caso, es la clase del componente la encargada de definir el formulario.
Nos ofrece la posibilidad de:

- Creación de los campos.
- Generación de reglas de validación.
- Posibilidad de tratar con los cambios que se realicen en los campos del formulario.
- Se pueden generar de una manera más cómoda pruebas unitarias sobre los datos de un formulario.

La diferencia con la anterior fórmula reside en la potencia de nuestros desarrollos, es un tipo de formulario que me ofrece muchas más cosas que el anterior modelo y mucha más versatilidad. Template Driven sin embargo, es mucho más rápida en su implementación.

Para trabajar con este tipo de formularios, debemos importar dentro de la definición de nuestro módulo `ReactiveFormsModule`.

```
import { NgModule } from '@angular/core';
import { ReactiveFormsModule } from '@angular/forms';
import { BrowserModule } from '@angular/platform-browser';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { PrimerComponentesComponent } from './components/primer-componentes/primer-componentes.component';

@NgModule({
  declarations: [
    AppComponent,
    PrimerComponentesComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    ReactiveFormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Es posible que en algunos proyectos necesitemos tener activos los dos módulos.

Trabajaremos con dos objetos básicos, **FormGroup** y **FormControl**, para generar grupos de controles y para identificar los controles en sí, respectivamente.
Vemos un ejemplo de clase en la que incorporamos la definición de un formulario.

```
import { Component, OnInit } from '@angular/core';
import { FormControl, FormGroup } from '@angular/forms';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
```

```
export class PrimerComponentesComponent implements OnInit {

  form: FormGroup;
  constructor() {
    this.form = new FormGroup({
      nombre: new FormControl(''),
      apellidos: new FormControl(''),
      edad: new FormControl(''),
      direccion: new FormControl('')
    })
  }

  ngOnInit() {
  }

}
```

El siguiente paso sería modificar nuestra plantilla HTML para poder trabajar con los campos que hemos definido dentro del objeto formulario.

Ya no necesitamos especificar la directiva `ngModel` para cada uno de los campos, pero sí debemos concretar con qué propiedad se enlazan de las definidas dentro del objeto.

Esto lo conseguimos gracias a la directiva `formControlName`.

Vemos un ejemplo de plantilla a partir de los campos anteriores.

```
<form novalidate [formGroup]="form" (ngSubmit)="onSubmit()">
  <input type="text" name="nombre" formControlName="nombre">
  <input type="text" name="apellidos" formControlName="apellidos">
  <input type="text" name="edad" formControlName="edad">
  <input type="text" name="direccion" formControlName="direccion">
  <button type="submit">Enviar</button>
</form>
```

El evento `onSubmit()` recogerá todos los datos del formulario y los sacará por consola, de la misma forma que el formulario anterior me devolverá un objeto cuyas claves serán cada uno de los `formControlName` que tiene cada elemento del formulario.

7.6. Validaciones en Angular y validaciones personalizadas

Los formularios en Angular soportan dos tipos de validaciones:

- Las built-in validations.
- Las validaciones personalizadas.

Las validaciones del primer tipo las encontramos definidas dentro de la clase **Validators** en el paquete forms de Angular.

En la definición de nuestros controles, podemos agregar los validadores que necesitemos para cada campo concreto, como segundo parámetro recibimos un array de validadores donde podemos tener la cantidad de validadores por campo que deseemos.

Vemos un ejemplo con el formulario anterior a partir de una expresión regular para el nombre y de un validador required para la edad.

```
import { Component, OnInit } from '@angular/core';
import { FormControl, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-primer-componente',
  templateUrl: './primer-componentes.component.html',
  styleUrls: ['./primer-componentes.component.css']
})
```

```
export class PrimerComponentesComponent implements OnInit {

  form: FormGroup;
  constructor() {
    this.form = new FormGroup({
      nombre: new FormControl('', [
        Validators.pattern('[\\w\\-\\s\\/]+'),
      ]),
      apellidos: new FormControl(''),
      edad: new FormControl('', [
        Validators.required
      ]),
      direccion: new FormControl('')
    })
  }

  ngOnInit() {

  }

  onSubmit(){
    console.log(this.form.value)
  }
}
```

Con la inclusión del segundo parámetro en la creación del objeto `FormControl` hemos conseguido que Angular detecte cuando el campo está incorrecto y en ese caso nos agrega al campo una nueva clase CSS llamada `ng-invalid`**.**

Si el campo es válido, elimina esa clase y coloca `ng-valid`.

Es tarea del desarrollador asegurarse que todos los campos están correctos antes de enviar el formulario. Angular **no se encarga de bloquear el envío.**

A partir de la instancia generada para manejar el formulario, podemos trabajar con las propiedades `valid / invalid`**.**

Si todos los campos cumplen con sus validaciones la propiedad `valid` obtiene el valor **true**.

Por lo tanto, podríamos controlar el envío del formulario con validaciones de este estilo:

```
<button type="submit" [disabled]="!form.valid">Enviar</button>
```

Dependiendo de la interacción que hayamos tenido con nuestros controles, vamos a ir viendo cómo se modifica su estado.

Aparte de los valores `valid/invalid` disponemos de otra serie de propiedades que nos permiten indicar qué estado tienen nuestros controles en cada momento.

Con `dirty` sabemos si el usuario ha modificado el valor del control o no. El caso contrario nos lo devuelve la propiedad `pristine`.

```
<pre>
 Dirty -> {{ form.controls.nombre.dirty }}
 Pristine -> {{ form.controls.nombre.pristine }}
</pre>
```

Obtendremos el valor true dentro de la propiedad `touched` si el usuario ha puesto el foco en un control y posteriormente pasa a otro.

Por ejemplo, cuando trabajamos sobre un campo y posteriormente, con el tabulador o haciendo **click** en otro control, cambiamos el foco.

La diferencia entre `touched` y `dirty` es que se puede activar la primera sin necesidad de modificar su valor.

El contrario es `untouched`.

```
<pre>
 Touched -> {{ form.controls.nombre.touched }}
 Untouched -> {{ form.controls.nombre.untouched }}
</pre>
```

Nota la etiqueta `<pre>` solo es para que se visualice en formato código por si queréis ponerlo a modo de comentario dentro del vuestros formularios, no significa absolutamente nada dentro del Angular

Si los validadores de un control concreto devuelve true, tendremos activa la propiedad `valid`.

El opuesto a esta propiedad es `invalid`.

```
<pre>
 Valid -> {{ form.controls.nombre.valid }}
 Invalid -> {{ form.controls.nombre.invalid }}
</pre>
```

Validadores Custom

Para crear nuestros propios validadores necesitamos generar un método que recibe como parámetro el control a validar y devuelve `null` si está correcto o un objeto especificando los errores si la validación es incorrecta.

Veamos un ejemplo de cómo podríamos validar si la edad en nuestro formulario se encuentra entre los 18 y los 65.

```
edadValidator(control) {
    if (control.value.trim().length === 0) return null
    let edad = parseInt(control.value)
    let minEdad = 18
    let maxEdad = 65
```

```
    if (edad >= minEdad && edad <= maxEdad) {
      return null
    } else {
      return { 'message': 'La edad debe estar entre 18 y 65' }
    }
  }
```

Este validador debemos ponerlo dentro del `formControlName` en el array de validadores del campo donde se aplique.

Ejemplo.

```
  this.form = new FormGroup({
      nombre: new FormControl('', [
        Validators.pattern('[\\w\\-\\s\\/]+'),
      ]),
      apellidos: new FormControl(''),
      edad: new FormControl('', [
        Validators.required,
        this.edadValidator
      ]),
      direccion: new FormControl('')
    })
```

Manejo de errores

Aparte de las propiedades anteriores, para cada control disponemos de la propiedad `errors`, dentro de la cual queda definido si dicho control pasa o no los validadores.

Vemos un ejemplo con los campos anteriores.

```
<input type="text" name="edad" formControlName="edad">
<div *ngIf="form.controls.nombre.errors">
  La edad es incorrecta
</div>
```

Se puede ser incluso más específico haciendo referencia al validador en concreto sobre el cual queramos llamar la atención.

```
<input type="text" name="nombre" formControlName="nombre">
<div *ngIf="form.controls.nombre.errors?.required">
  El nombre es un campo requerido
</div>
```

El símbolo «?» indica que se va a validar la propiedad `required` siempre y cuando `errors` sea diferente de `null`.

Para nuestros validadores personalizados, podemos acceder al objeto que devolvemos en caso de que no se valide el valor.

```
<input type="text" name="edad" formControlName="edad">
<div *ngIf="form.controls.edad.errors">
  {{ form.controls.edad.errors?.message }}
</div>
```

Y así es como podemos gestionar las validaciones gráficamente dentro de nuestros formularios de tipo Model para lanzar los errores del usuario a nuestro interfaz.

A fondo

Introducción al servicio e inyección de dependencias en Angular

Documentación Angular. https://docs.angular.lat/guide/architecture-services

Explicación extensa y oficial de cómo se gestionan la inyección de dependencias en angular.

Diferencias entre formularios de tipo template y model

Mugika, A. (14 de enero de 2021). *Formularios en Angular – Diferencias Template y Reactive Forms* [Blog]. https://mugan86.medium.com/formularios-en-angular-diferencias-template-y-reactive-forms-e37af5e30b81

Interesante articulo donde se explican las diferencias entre los dos formularios desde otro punto de vista.

Documentación oficial formularios en español Angular

Documentación Angular. https://docs.angular.lat/start/start-forms

Explicación extensa y oficial de cómo se gestionan formularios en Angular.